Project Euler
C Implementations: Methods and Corrections

Source-based technical commentary

November 2026

Abstract

This document covers the 37 Project Euler problems represented in the KO0I/projectEuler repository, in ascending problem order. The original explanations and assessments describe historical C attempts, including unfinished programs and defects; their source links point to the preserved reorganization branch. They must not be mistaken for assessments of the subsequently corrected implementations.

Reading the corrections: Red, explicitly labelled sections explain the repaired or completed C implementation. The main C files now own their algorithms and call reusable mathematical helpers in `support/c/euler_common.c`. Historical programs remain under `solutions/c/variants/historical/`. Problems 13 and 17 have new C implementations as well as their preserved earlier analyses.

All 37 C answers, 37 Rust answers and 37 Forth answers passed the same-machine validation baseline recorded in `docs/VALIDATION.md`. Each program was run once in that profile. Passing the fixed reference inputs is evidence of correctness for those inputs, not a proof of a general-purpose algorithm. The later 120-entry Gforth run recorded in `docs/VALIDATION_FORTH_1_120.md` passes 115 entries with a 60-second limit per solver. The remaining five are timeouts; the run contains no wrong answers or runtime errors.

Contents

1 Multiples of 3 or 5 — Problem 1 **4**

2 Even Fibonacci Numbers — Problem 2 **5**

3 Largest Prime Factor — Problem 3 **6**

4 Largest Palindrome Product — Problem 4 **7**

5 Smallest Multiple — Problem 5 **8**

6 Sum Square Difference — Problem 6 **9**

7 10 001st Prime — Problem 7 **10**

8 Largest Product in a Series — Problem 8 **11**

9 Special Pythagorean Triplet — Problem 9 **12**

10 Summation of Primes — Problem 10 **13**

11 Largest Product in a Grid — Problem 11 **14**

12 Highly Divisible Triangular Number — Problem 12 **15**

13 Large Sum — Problem 13 **16**

14 Longest Collatz Sequence — Problem 14 **17**

15 Power Digit Sum — Problem 16 **18**

16 Number Letter Counts — Problem 17 **19**

17 Counting Sundays — Problem 19 **20**

18 Factorial Digit Sum — Problem 20 **21**

19 Amicable Numbers — Problem 21 **22**

20 Names Scores — Problem 22 **23**

21 1000-digit Fibonacci Number — Problem 25 **24**

22 Quadratic Primes — Problem 27 **25**

23 Number Spiral Diagonals — Problem 28 **26**

24 Distinct Powers — Problem 29 **27**

25 Digit Fifth Powers — Problem 30 **28**

26 Digit Factorials — Problem 34 **29**

27 Circular Primes — Problem 35 **30**

28 Integer Right Triangles — Problem 39 **31**

29 Champernowne's Constant — Problem 40 **32**

30 Pentagon Numbers — Problem 44 **33**

31 Goldbach's Other Conjecture — Problem 46 **34**

32 Distinct Primes Factors — Problem 47 **35**

33 Self Powers — Problem 48 **36**

34 Consecutive Prime Sum — Problem 50 **37**

35 Lychrel Numbers — Problem 55 **38**

36 Totient Maximum — Problem 69 **39**

37 Square Digit Chains — Problem 92 **40**

38 Cross-language algorithm comparisons **41**

38.1 Problem 9: Special Pythagorean Triplet 41

38.2 Problem 11: Largest Product in a Grid 42

38.3 Problem 28: Number Spiral Diagonals 42

38.4 Problem 39: Integer Right Triangles 43

38.5 Exact arithmetic: Factor versus C and Gforth 44

38.6 Profiling interpretation 44

39 Mathematical visualization atlas **46**

39.1 Integer geometry and spirals 46

39.2 Grids and dynamic programming 46

39.3 Discrete counting and networks 46

1 Multiples of 3 or 5 — Problem 1

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/multiples_of_3_or_5_1.c`

Assessment: Correct for the specified bound.

Mathematical goal

The problem asks for the sum of integers below 1000 divisible by three or five:

$$S = \sum_{n=1}^{999} n \, \mathbf{1}_{(3|n)\vee(5|n)}.$$

The disjunction counts multiples of both divisors once.

How the C code works

The loop variable `i` runs from 0 to 999. The remainder tests `i%3==0` and `i%5==0` select the eligible integers. The logical OR combines the conditions; the accumulator `sum` starts at zero and receives each selected integer exactly once. The result is printed with `printf`.

Correctness and limitations

The code is correct for the stated bound and produces `...3168`. Including zero has no effect. The procedure takes $O(N)$ time and $O(1)$ auxiliary space. Inclusion–exclusion would be faster for very large limits, but it is not used here.

Correction to historical C — Problem 1

Correction: The existing direct enumeration already expressed the desired disjunction correctly. The corrected code retains this method and emits only the numeric answer.

2 Even Fibonacci Numbers — Problem 2

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/even_fibonacci_numbers_2.c`

Assessment: Undefined behavior.

Mathematical goal

The Fibonacci recurrence is $F_n = F_{n-1} + F_{n-2}$. The required sum includes even terms at most four million:

$$S = \sum_{\substack{F_n \le 4\,000\,000 \\ 2 \mid F_n}} F_n.$$

How the C code works

The program allocates `fib[33]`, initializes the first two entries to one and fills the array using `fib[i-2]+fib[i-1]`. A second loop checks evenness with `fib[k]%2==0`, accumulating qualifying entries in `sum`. This separates generating the sequence from filtering it.

Correctness and limitations

The declared `unsigned long sum` is never initialized before `sum += fib[k]`, so the behavior is undefined and the output cannot be trusted. It should start at zero. A value-bound loop would also be clearer than a fixed term count. The intended mathematical result is `...3732`, not a reliable result of this source.

Correction to historical C — Problem 2

Correction: The original accumulator was never initialized. The corrected routine begins with zero and generates Fibonacci terms until their value exceeds four million, rather than trusting a fixed array length.

3 Largest Prime Factor — Problem 3

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_prime_factor_3.c`

Assessment: Core method implemented.

Mathematical goal

The goal is to determine the largest prime factor of 600851475143. Removing factors one by one isolates the greatest factor remaining at the end.

How the C code works

The function `factorize` starts with candidate divisor `d=2`. If `n%d==0`, it divides `n` by `d` and tries the same divisor again, removing repeated factors. Otherwise it moves to 3 and then successive odd candidates. Once the loop condition `d<n` becomes false, it prints `d`.

Correctness and limitations

This factor-stripping approach yields the expected factor `...6857` for the supplied value. `unsigned long` needs to be at least 64 bits to represent the input portably. Testing candidates only up to the square root of the remaining number could improve efficiency.

Correction to historical C — Problem 3

Correction: The corrected trial-division routine uses a 64-bit integer and stops factoring once a candidate divisor exceeds the square root of the remaining number.

4 Largest Palindrome Product — Problem 4

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_palindrome_product_4.c`

Assessment: Dependent on generated input.

Mathematical goal

Find the maximum ab for three-digit integers a, b such that the decimal digits of ab form a palindrome.

How the C code works

The program opens `palindrome`, reads each candidate number into `temp`, and tests `a` from 100 to 999. If `temp%a==0`, it tests `b` in the same range and checks `a*b==temp`. A successful product greater than the running `max` replaces it. The CPU time is measured using `clock`.

Correctness and limitations

The source assumes the input file contains a complete, correctly separated list of palindromes. The historical palindrome generator does not reliably separate output records, so the data path must be verified. The calculated quotient `q=temp/a` is unused; checking whether `q` is a three-digit integer would remove the inner loop. The expected maximum is `...6609`, but it is not verified by the existing file workflow.

Correction to historical C — Problem 4

Correction: The new implementation enumerates products of three-digit integers and tests each product as a palindrome. It no longer relies on the historical palindrome input file or an unused second factor search.

5 Smallest Multiple — Problem 5

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/smallest_multiple_5.c`

Assessment: Logically valid but extremely slow.

Mathematical goal

The desired integer is $\text{lcm}(1, 2, \ldots, 20)$: the least positive value divisible by every integer from 1 to 20.

How the C code works

The variable `mult` begins at one. The `for` loop tests `mult%i` for each divisor up to `limit=20`. Whenever a test fails, `mult` increases and the divisor index is reset so that the new candidate is checked again. After all divisibility tests pass, the code prints the candidate.

Correctness and limitations

The first qualifying candidate is `...2560`, but checking essentially every preceding integer is computationally expensive. A much faster implementation would multiply the highest prime powers not exceeding 20. The surrounding `done` loop is not needed to express the search.

Correction to historical C — Problem 5

Correction: Unbounded candidate-by-candidate searching is replaced by successive least common multiples computed using the Euclidean greatest common divisor.

6 Sum Square Difference — Problem 6

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/sum_square_difference_6.c`

Assessment: Correct for the input.

Mathematical goal

For $N = 100$ compute

$$\left(\sum_{k=1}^{N} k\right)^2 - \sum_{k=1}^{N} k^2.$$

How the C code works

The helper `squareSum` adds the integers from zero through N before squaring the accumulated total. The helper `sumSquare` squares each integer individually and adds the squares. `main` subtracts the results and prints the intermediate quantities and difference.

Correctness and limitations

The algorithm computes `...4150` for $N = 100$. It takes $O(N)$ time and constant extra space. Closed-form expressions for the two sums would make the calculation $O(1)$, and wider integer types would allow larger N.

Correction to historical C — Problem 6

Correction: The corrected code evaluates the exact closed forms for the ordinary sum and the sum of squares. The original pair of accumulation loops was also mathematically sound for the specified input.

7 10 001st Prime — Problem 7

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/10_001st_prime_7.c`

Assessment: Does not reliably solve the named problem.

Mathematical goal

The aim is to identify p_{10001}, the prime in position 10001 of the ordered sequence $2, 3, 5, 7, \ldots$.

How the C code works

The program seeds a table with 2 and 3, examines successive odd candidates using trial division up to a rounded square root, and writes accepted candidates to a file named `primes`. Comments discuss using the table for other Euler problems as well.

Correctness and limitations

The hard-coded `countDone=78497` is unrelated to the 10001st prime, and the desired table entry is never printed. Both the generation loop and the output loop access index `countDone` in an array of that length, which is out of bounds. Candidate-reset behavior during trial division also needs correction. The expected prime is `...4743`; this file does not establish it.

Correction to historical C — Problem 7

Correction: The array bound overflow and unrelated prime-count target are removed. The corrected search counts primes until the 10001st candidate is found.

8 Largest Product in a Series — Problem 8

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_product_in_a_series_8.c`

Assessment: Unimplemented.

Mathematical goal

Given the decimal digits d_i, find a window of the prescribed length w maximizing $\prod_{j=0}^{w-1} d_{i+j}$.

How the C code works

The main file contains only two header inclusions. It defines no input array, search loop, product calculation, or `main` function. A historical variant in `solutions/c/variants/code/` records manual observations about products of large digits, but does not implement the algorithm.

Correctness and limitations

This is a placeholder rather than a runnable solution. A complete implementation would scan every length-w window, compute the product, and maintain the maximum. A rolling product is possible with special handling for zero digits.

Correction to historical C — Problem 8

Correction: The historical main C file was only a stub. The completed algorithm examines every 13-digit window of the exact 1000-digit input and retains the largest product.

9 Special Pythagorean Triplet — Problem 9

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/special_pythagorean_triplet_9.c`

Assessment: Search implemented.

Mathematical goal

Find positive integers $a < b < c$ satisfying $a^2 + b^2 = c^2$ and $a + b + c = 1000$, then calculate abc.

How the C code works

The outer loop enumerates `a` and the inner loop enumerates `b`. Rather than search all three values independently, it computes `c=sum-a-b`, making the perimeter constraint automatic. It tests `a*a+b*b==c*c`. A match sets `found` and prints the triplet and product.

Correctness and limitations

The search finds $(a, b, c) = (200, 375, 425)$, giving `...5000`. The `break` exits only the inner loop, so the outer loop continues until its bound before the enclosing `while` stops. A no-solution case would repeat forever because there is no explicit failure exit.

Correction to historical C — Problem 9

Correction: The perimeter-constrained nested search is retained, but the corrected routine terminates immediately after finding the valid triple.

10 Summation of Primes — Problem 10

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/summation_of_primes_10.c`

Assessment: Computes a different sum.

Mathematical goal

The problem asks for the sum of primes strictly below two million:

$$S = \sum_{\substack{p < 2\,000\,000 \\ p \text{ prime}}} p.$$

How the C code works

The program reads an integer `n` from its command-line argument and loops from `i=1` through `n`, adding each integer to `sum`. It thus computes the arithmetic series $n(n+1)/2$. There is no primality test.

Correctness and limitations

This C program does not solve the prime-summation problem. Using `int` also overflows for the intended bound. A sieve of Eratosthenes combined with a 64-bit accumulator would solve the actual problem. The mathematical answer is `...8922`, not the sum produced by the present algorithm.

Correction to historical C — Problem 10

Correction: The old program summed all integers, not primes, and used an undersized accumulator. The corrected implementation uses a sieve of Eratosthenes and a 64-bit prime sum.

11 Largest Product in a Grid — Problem 11

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_product_in_a_grid_11.c`

Assessment: Undefined indexing in diagonal loops.

Mathematical goal

Find the largest product of four adjacent entries along any row, column, or diagonal in the fixed 20×20 grid.

How the C code works

The code stores the grid as `long grid[20][20]` and makes four directional sweeps. For each starting location it multiplies four entries and compares the result with `max`. The horizontal and vertical sweeps use appropriate bounds for their directions.

Correctness and limitations

Both diagonal expressions contain `j-1`, `j-2`, and `j-3` even when `j` begins at zero, producing negative indices and undefined behavior. The diagonal starting-column bounds must be repaired. The unused macro `#define N = 4;` is not a valid numeric definition for use in expressions. The known maximum is `...0674` but is not safely demonstrated by this source.

Correction to historical C — Problem 11

Correction: The old diagonal sweeps accessed negative column indices. The corrected implementation checks all four directions with explicit endpoint bounds before multiplying grid entries.

12 Highly Divisible Triangular Number — Problem 12

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/highly_divisible_triangular_number_12.c`

Assessment: Partial: triangular numbers only.

Mathematical goal

A triangular number is $T_n = n(n+1)/2$. The challenge seeks the first one with more than 500 positive divisors.

How the C code works

Starting from `tri=1`, the loop adds every integer from 2 through 500. The printed value is $T_{500} = 125250$. The variable `j` is incremented but has no effect on the result.

Correctness and limitations

The source never counts the divisors of a triangular number and never tests a divisor-count threshold, so it does not solve the full problem. Efficient factorization of n and $n + 1$, with their known coprimality, can support a complete divisor-count search.

Correction to historical C — Problem 12

Correction: The original source only computed the 500th triangular number. The corrected search generates successive triangular numbers and explicitly counts their positive divisors until the threshold is exceeded.

13 Large Sum — Problem 13

Source: `https://github.com/KOOI/projectEuler/blob/feature/euler-c-rust-forth/solutions/c/large_sum_13.c`

Mathematical goal

Sum the one hundred provided 50-digit integers, then report the first ten decimal digits of their exact sum.

Historical approach

The original repository contains a short shell script attempting to sum file rows using `awk`. Floating-point addition cannot reliably retain every digit of this sum.

Corrected C implementation

Correction: The C solution reads each value from the versioned `lsum` input, accumulates it exactly with GNU MP arbitrary-precision integers, and extracts the leading ten decimal digits. The historical shell attempt remains preserved.

Validation

The C, Rust and Forth implementations all returned `...6230` for the repository's original 100-number input in the complete CI validation baseline.

14 Longest Collatz Sequence — Problem 14

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/longest_collatz_sequence_14.c`

Assessment: Search implemented; reporting format unsafe.

Mathematical goal

For every starting integer $2 \leq n < 10^6$, iterate $n \mapsto n/2$ when even, or $n \mapsto 3n + 1$ when odd. Find the start whose chain reaches 1 after the greatest number of terms.

How the C code works

The helper `collatz` counts the current number as the first term and advances until reaching 1. `main` compares every length with `max` and records the corresponding start in `maxi`. It also prints each candidate's length.

Correctness and limitations

The implementation redundantly calls `collatz(i)` up to three times. The final `printf` uses `%i` for `unsigned long long` values, causing undefined behavior; use `%llu`. Intermediate values can exceed the starting limit and may overflow. The expected starting value is `...7799`, but the final formatting must be repaired.

Correction to historical C — Problem 14

Correction: The corrected implementation calculates each chain length only once per starting value and returns the winning start without the original mismatched printf conversion specifiers.

15 Power Digit Sum — Problem 16

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/power_digit_sum_16.c`

Assessment: GMP-based algorithm implemented.

Mathematical goal

Compute the sum of the base-ten digits of 2^{1000}. GNU MP arbitrary-precision integers allow the power to be represented exactly.

How the C code works

The call `mpz_pow_ui` computes 2^{1000}. At each iteration the code calculates `giant mod a` where `a` runs through powers of ten, then divides by `a/10` to isolate the corresponding digit. The digit is added into the GMP variable `result`, which is initialized to zero by `mpz_init`.

Correctness and limitations

The 305 iterations exceed the 302-digit input, so later positions contribute zero. The expected sum is `...1366`. Compilation requires GMP headers and `-lgmp`. A decimal-string conversion would be simpler, but it is not the method implemented here.

Correction to historical C — Problem 16

Correction: GNU MP is retained for exact exponentiation, but the corrected code converts the resulting integer to decimal text and sums digit values directly.

16 Number Letter Counts — Problem 17

Source: `https://github.com/KOOI/projectEuler/blob/feature/euler-c-rust-forth/solutions/c/number_letter_counts_17.c`

Mathematical goal

For integers 1 through 1000 inclusive, count letters in their British English names, excluding spaces and hyphens; non-round hundreds use the word *and*.

Historical approach

The original LaTeX analysis at `solutions/latex/number_letter_counts_17.tex` counts recurring words mathematically. There was no corresponding main C implementation.

Corrected C implementation

Correction: The new C solution tabulates the lengths of words for one through nineteen and the multiples of ten, adds seven letters for *hundred* and three for *and* where applicable, handles *one thousand* separately, and sums across all 1000 numbers.

Validation

The corrected C code matched the expected value `...1124` in local testing.

17 Counting Sundays — Problem 19

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/counting_sundays_19.c`

Assessment: Calendar simulation implemented.

Mathematical goal

Count months whose first day is Sunday between January 1901 and December 2000, inclusive.

How the C code works

The global state tracks year, month `j`, weekday `i`, day of month and `score`. It starts on Tuesday, January 1, 1901. `nextDay` cycles through weekdays, and `nextMonth` advances to the first of the next month and increments `score` when that day is Sunday. The main loop handles February and months with 30 or 31 days.

Correctness and limitations

The test `year%4==0` omits the full Gregorian century rule, but covers leap years in the specified 1901–2000 interval, including 2000. Formal function arguments are unused, and the stopping boundary deserves tests. The intended answer is `...171`.

Correction to historical C — Problem 19

Correction: The corrected calendar traversal uses the full Gregorian leap-year rule and counts Sundays at the start of each month only within the requested date interval.

18 Factorial Digit Sum — Problem 20

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/factorial_digit_sum_20.c`

Assessment: GMP digit sum implemented; output label wrong.

Mathematical goal

Find the sum of the decimal digits of 100!.

How the C code works

The call `mpz_fac_ui(factorial,100)` constructs the factorial exactly. A modulus with powers of ten retains the lowest digits; division by the next lower power isolates one digit. Each digit is added to `result`, an arbitrary-precision accumulator.

Correctness and limitations

The loop runs beyond the decimal length, but the surplus digits are zero. The expected sum is `...648`. The output message incorrectly describes digits of 2^{100}, copied from problem 16; it should name 100!. Compiling requires GMP and `-lgmp`.

Correction to historical C — Problem 20

Correction: GNU MP still computes 100 factorial exactly. The corrected digit-sum routine eliminates the copied message that incorrectly described a power of two.

19 Amicable Numbers — Problem 21

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/amicable_numbers_21.c`

Assessment: Pair-validation logic needs correction.

Mathematical goal

Let $d(n)$ denote the sum of positive proper divisors of n. An amicable pair has $d(a) = b$, $d(b) = a$, and $a \neq b$. Sum the amicable numbers under 10000.

How the C code works

The helper `d` tries divisors from 2 through $n - 1$, beginning its sum at one. The `amicable` loop computes `a=d(j)` and `b=d(a)` for each candidate `j`, then adds `a+b` when its tests pass and the total differs from the previous pair sum.

Correctness and limitations

The condition `d(a)==b` is true by construction, but the source never requires `b==j`. The candidate seed therefore need not belong to the pair being added. `prevsum` only filters consecutive duplicates, and partner bounds are not enforced. The correct total is `...1626`; the code needs stronger pair checks before claiming it.

Correction to historical C — Problem 21

Correction: Each candidate below the requested limit is counted only when it has a distinct proper-divisor-sum partner that maps back to the candidate. The earlier consecutive-sum deduplication is removed.

20 Names Scores — Problem 22

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/names_scores_22.c`

Historical assessment: Full workflow attempted; parser fragile.

Mathematical goal

Alphabetize the names in the input file, sum each name's letter values $A = 1, \ldots, Z = 26$, multiply by its one-based sorted position, and sum all the scores.

How the historical C code works

The file `names.txt` is parsed into a fixed array of 5163 names. Several adjacent-swap sorting passes examine successive character positions. After sorting, uppercase ASCII letters are valued by subtracting 64, each name score is multiplied by `j+1`, and the grand total is accumulated.

Historical correctness and limitations

Parsing by byte positions and fixed name lengths is fragile; the multi-pass sort should be replaced by lexicographic comparison with `strcmp` and `qsort`. On input failure the old code calls `fclose` on a null pointer. The expected score is `...8282`, but the old program should not be assumed correct merely because the reference answer is known.

Correction to historical C — Problem 22

Correction: The fixed-pass string sort and fragile byte seeking are replaced with bounded parsing of the quoted data, standard lexicographic sorting, and positional scoring. The corrected C implementation, independent Rust implementation and Forth implementation each passed the expected answer `...8282` using the original repository dataset of 5163 names.

21 1000-digit Fibonacci Number — Problem 25

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/1000_digit_fibonacci_number_25.c`

Assessment: GMP search with indexing and portability defects.

Mathematical goal

Find the first Fibonacci index n with $F_n \geq 10^{999}$, which is equivalent to reaching 1000 decimal digits.

How the C code works

GMP integers `a` and `b` start at one. The nested helper `fibonacci` adds these numbers and swaps them to advance the sequence. The threshold `top=10^999` is constructed with `mpz_ui_pow_ui`. A quotient `lentest=a/top` is compared with one to detect the first term above the threshold. A counter `i` increments for each generated term.

Correctness and limitations

A nested function is a GNU C extension, not ISO C. Because the initial two sequence terms are already present, the counter starting at zero does not equal the standard Fibonacci index. Several `printf` calls use `%i` for `unsigned long` values. The memory cleanup is placed after an unreachable `return`. The correct index is `...4782`; the program's counter must be adjusted before treating it as that answer.

Correction to historical C

Correction: The corrected GNU MP recurrence advances the standard Fibonacci index until the value has 1000 digits. It removes the historical indexing and unreachable cleanup errors.

22 Quadratic Primes — Problem 27

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/quadratic_primes_27.c`

Assessment: Exhaustive search with primality edge cases.

Mathematical goal

Find coefficients a, b with $|a| < 1000$, $|b| \leq 1000$, maximizing the length of the prime-producing run of $f(n) = n^2 + an + b$ for $n = 0, 1, \ldots$. Report ab.

How the C code works

Two nested loops enumerate coefficient pairs. The helper `ipow` computes the square, and `isPrime` decides whether each evaluated polynomial value continues the run. The `score` increases while values appear prime. The first nonprime ends the run, updates `highscore` and the best coefficient pair if needed, and resets `n` and `score`.

Correctness and limitations

The local primality checker misclassifies 2 as nonprime, because its divisor loop does not execute and its default return value denotes nonprime. It calls `sqrt` before handling negatives. The algorithm also explores many coefficients with b nonprime, although $b = f(0)$ must be prime for a nonempty run. The expected coefficient product is `...9231`; it should be verified only after repairing the primality test.

Correction to historical C

Correction: The primality test now correctly accepts two and excludes negative and nonprime values, including zero and one. The coefficient search starts at n equals zero.

23 Number Spiral Diagonals — Problem 28

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/number_spiral_diagonals_28.c`

Assessment: Closed-form idea, but no accumulated sum.

Mathematical goal

Determine the total of both diagonals in a 1001×1001 clockwise number spiral. Each successive odd-width ring contributes four corner values.

How the C code works

Four functions, `A` through `D`, attempt to express the corner sequences algebraically. `main` chooses a width `n=5`, calls the four functions, adds the central one, and prints the result, thereby avoiding a full matrix representation.

Correctness and limitations

Every helper overwrites its global `sumA`, `sumB`, `sumC` or `sumD` on each loop iteration instead of adding a sequence of terms. The width is five, not 1001. The print statement uses integer `%ld` conversions with `long double` arguments, which is undefined behavior. The expected total is `...1001`, but this source does not compute it.

Correction to historical C

Correction: All four corners of every odd-width ring are accumulated through width 1001. The old code only retained the last term and used invalid floating-point print conversions.

24 Distinct Powers — Problem 29

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/distinct_powers_29.c`

Assessment: Diagnostic power table, not uniqueness count.

Mathematical goal

Count the distinct integers among a^b with $2 \leq a \leq 100$ and $2 \leq b \leq 100$. Different pairs can produce the same number, for example $4^3 = 8^2$.

How the C code works

The helper `ipow` uses exponentiation by squaring: it multiplies the accumulated result when an exponent bit is set, shifts the exponent, and squares the base. `main` prints a small table for bases and exponents 2 through 6. This demonstrates power generation but not deduplication.

Correctness and limitations

No distinct-values set or counter is present. A signed `int` cannot represent general powers up to 100^{100}, though the small demonstration fits. A corrected algorithm could normalize bases to primitive powers and count exponent combinations, or use GMP values and a set. The expected distinct count is `...9183`, not an output of the existing program.

Correction to historical C

Correction: The former sample table is replaced by exact GNU MP powers that are sorted and deduplicated to count distinct results.

25 Digit Fifth Powers — Problem 30

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/digit_fifth_powers_30.c`

Assessment: Direct exhaustive solution.

Mathematical goal

Find nontrivial positive integers whose value equals the sum of the fifth powers of their decimal digits, then sum those integers.

How the C code works

The helper `powi` evaluates integer powers by repeated multiplication. For each candidate `j` from 2 to one million, seven expressions using division and remainder extract decimal digits from units through millions. Their fifth powers are summed and compared to `j`. Matching candidates are printed and added to `answer`.

Correctness and limitations

This search covers all candidates that can meet the condition, and leading zero digits contribute zero. The expected sum is `...3839`. The upper bound is conservative; using the maximum digit-power sum gives a tighter bound. The unused variable `k` and outdated comment about fourth powers do not change the computation.

Correction to historical C

Correction: The original digit-power test is retained with a mathematically bounded search domain rather than an arbitrary ceiling.

26 Digit Factorials — Problem 34

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/digit_factorials_34.c`

Assessment: Direct search with an unjustified upper bound.

Mathematical goal

Find numbers equal to the sum of the factorials of their decimal digits and sum them, excluding the trivial cases 1 and 2.

How the C code works

The helper `fact` calculates a digit's factorial by multiplying integers from one through that digit, with special cases for zero and negative input. The outer loop tests `j` from 100 through 99999. Remainder and division expressions isolate the units through ten-thousands digits, and the code sums their factorials into `t`. If `t==j`, the candidate contributes to the final `sum`.

Correctness and limitations

This range happens to include 145 and 40585, whose total is `...0730`, but the source does not prove that 99999 is a sufficient upper bound. A rigorous bound comes from comparing a d-digit number with $d \cdot 9!$; candidates of six or seven digits cannot be dismissed merely by intuition. The program should derive its stopping criterion and avoid recalculating the same digit factorials.

Correction to historical C

Correction: The search now covers every candidate up to seven times nine factorial, instead of the unjustified five-digit ceiling.

27 Circular Primes — Problem 35

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/circular_primes_35.c`

Assessment: Unfinished scaffolding.

Mathematical goal

A prime is circular when every cyclic rotation of its decimal digits is also prime. Count all circular primes below one million.

How the C code works

The source describes separate ranges by digit length and sketches functions `a`, `b`, `c` and `d` to examine rotations. The actual `main` sets up a few variables, opens `circprimes`, checks for file-open failure, and closes it. No rotations are calculated and no answer is printed.

Correctness and limitations

The helper functions have empty bodies despite declaring integer return types. The initialized `sum=9` is neither updated nor printed. A complete solution needs prime membership tests for every rotation, with attention to leading zeros and repeated cyclic forms. The known count is `...55`, not a result established by this placeholder.

Correction to historical C

Correction: The empty historical helper stubs have been replaced with a complete sieve and cyclic digit-rotation search for every prime below one million.

28 Integer Right Triangles — Problem 39

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/integer_right_triangles_39.c`

Assessment: Restricted search with counting flaws.

Mathematical goal

For perimeters $p \leq 1000$, count integer-sided right triangles $a < b < c$ with $a^2 + b^2 = c^2$ and $a + b + c = p$. Determine which perimeter has the most distinct triples.

How the C code works

The global `p` begins at 12. Nested loops search restricted intervals of sides `a`, `b` and `c`, while `testvalid` checks the Pythagorean equation using `pow` and the perimeter sum. Each hit increases `score`. After a perimeter is tested, `highscore` may be updated; the code advances to another `p` and measures CPU time.

Correctness and limitations

The code advances `p` by four and later by eight, which skips many candidate perimeters. The side-loop bounds are interdependent and can omit triples; incrementing `c` after a match can skip additional triples. `testvalid` lacks a return for one possible branch, and the program prints the highest score rather than its perimeter. The expected winning perimeter is `...840`, but this search is not a reliable proof.

Correction to historical C

Correction: The corrected algorithm examines all perimeters up to 1000 without skipping candidates and reports the maximizing perimeter.

29 Champernowne's Constant — Problem 40

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/champernownes_constant_40.c`

Assessment: Partial digit-generation experiment.

Mathematical goal

Champernowne's constant is the concatenation 0.12345678910111213…. The task asks for the product of digits at specified powers-of-ten positions.

How the C code works

The program allocates an integer digit array of length `end=\texttt{...210}`. Separate branches attempt to fill the array with one-, two- and three-digit numbers, updating counters `secondpow` and `thirdpow` as the decimal length changes. At the end it prints the digit positions and the generated string.

Correctness and limitations

The program does not calculate the requested digit product or reach the millionth position. Its outer loop permits `n==end` while the later branches write `d[n+1]` or `d[n+2]`, causing out-of-bounds access. It also uses a custom index convention that needs verification against the first few concatenated digits. The historical `v2` alternative is likewise exploratory. The target product is `...210`, not a confirmed result of this code.

Correction to historical C

Correction: The corrected implementation generates decimal digits until position one million and multiplies the required digit values without a too-small array.

30 Pentagon Numbers — Problem 44

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/pentagon_numbers_44.c`

Assessment: Search has a repeated-condition bug.

Mathematical goal

Pentagonal numbers satisfy $P_n = n(3n-1)/2$. Find two pentagonal numbers whose sum and absolute difference are both pentagonal, minimizing their positive difference.

How the C code works

The helper `P` evaluates the formula, and `isPentagon` searches a precomputed array for an exact match, returning zero on success. The program generates 5000 pentagonal numbers, checks all ordered pairs in nested loops, and tracks a variable `D` initialized to 100.

Correctness and limitations

Both nested conditionals test `testdif`; neither tests `testsum`. As written, the key requirement that the sum is pentagonal is missing. The initial bound `D=100` also prevents recording a valid larger difference. The double loop is quadratic in the number of candidates, each with a linear scan for membership. The known minimum is `...2660`; it is not computed correctly by this source.

Correction to historical C

Correction: Both the sum and difference must be pentagonal; the revised search checks both predicates and minimizes a positive difference without the erroneous initial bound.

31 Goldbach's Other Conjecture — Problem 46

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/goldbachs_other_conjecture_46.c`

Assessment: Conceptual search, unsafe bounds and missing declarations.

Mathematical goal

The conjecture under test is that every odd composite integer z can be written $z = p + 2m^2$ for a prime p and positive integer m. Find the least counterexample.

How the C code works

The program reads prime numbers from `million_primes`, generates odd composite candidates using the external helper `isPrime`, then searches pairs (p, m) for the required decomposition. It calls external `powi` to square `m`, advances to the next candidate when a representation is found, and times the search.

Correctness and limitations

The source does not declare its external helper prototypes or link them directly; it must be built with `support/c/functions/isPrime.c` and `powi.c`. It reads without guarding the million-element array capacity and loops using `i<=1000000`, an out-of-range index. Advancing `i` inside nested loops can also invalidate the current candidate. The expected counterexample is `...5777`; the current implementation cannot be regarded as a safe computation of it.

Correction to historical C

Correction: The corrected odd-composite search tests prime-plus-twice-a-square representations directly, eliminating unsafe prime-file indexing and nested-loop index mutations.

32 Distinct Primes Factors — Problem 47

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/distinct_primes_factors_47.c`

Assessment: Search implemented with external factor counter.

Mathematical goal

Find the first four consecutive integers each having exactly four distinct prime divisors. For example, repetitions of the same prime count once.

How the C code works

The main loop tests starts from `i=4` upward. For each candidate start, it checks whether `distinctpfactors(i)` through `distinctpfactors(i+3)` all equal `score=4`. If all four tests pass, the program prints the sequence and exits the loop. It asserts that 60 has three distinct prime factors as a basic check.

Correctness and limitations

The function `distinctpfactors` lives in `support/c/functions/distinctpfactors.c` and is not declared in this source. Linking and a prototype are required; the helper's fixed-size factor array and counting logic also merit tests. The search limit of twenty million is arbitrary. The expected first integer is `...4043`, but the exact code and helper should be tested before certifying the result.

Correction to historical C

Correction: Prime factors are divided out completely before counting distinct factors, then four consecutive four-factor integers are sought.

33 Self Powers — Problem 48

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/self_powers_48.c`

Assessment: Modular computation implemented.

Mathematical goal

Compute the last ten decimal digits of $\sum_{n=1}^{1000} n^n$. Only the residue modulo 10^{10} is needed:

$$S \bmod 10^{10} = \left(\sum_{n=1}^{1000} n^n\right) \bmod 10^{10}.$$

How the C code works

GMP constructs the modulus `digitspan=10^10`. The loop evaluates each `n^n` with `mpz_ui_pow_ui`, reduces it modulo `digitspan`, and adds it into `result`. Finally the accumulated sum is reduced modulo the same modulus and printed.

Correctness and limitations

The loop runs from 999 down to 1, omitting the 1000th term. This omission does not change the last ten digits because 1000^{1000} is divisible by 10^{10}. The expected residue is `...6700`. The code computes full huge powers before reducing them; modular exponentiation would be significantly faster. GMP headers and `-lgmp` are required.

Correction to historical C

Correction: The corrected modular GNU MP calculation includes the term for 1000, which was previously omitted but happens to contribute zero modulo ten to the tenth power.

34 Consecutive Prime Sum — Problem 50

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/consecutive_prime_sum_50.c`

Assessment: Unsafe exploratory attempt.

Mathematical goal

Find the prime below one million that can be expressed as a sum of the greatest number of consecutive primes.

How the C code works

The program loads an initial segment of primes from `million_primes` into `primeList` and contains a loop intended to sum consecutive windows from an offset until a demonstration sum is reached. A local `isPrime` helper and result-tracking variables are present, but the main primality and scoring blocks are commented out.

Correctness and limitations

The array has only ten elements while the window loop reads at least 22, causing out-of-bounds access. `test` is read before initialization in the enclosing condition, and `fgets(line,8,fp)` can write eight bytes into a seven-byte array. A file-open failure is not handled. The program does not produce a valid answer; the expected prime is `...7651`. The alternative versions in `variants` are separate attempts.

Correction to historical C

Correction: A sieve and prefix sums replace the historical ten-element array, its out-of-bounds reads, and the uninitialized accumulator.

35 Lychrel Numbers — Problem 55

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/lychrel_numbers_55.c`

Assessment: Reverse-add search limited by integer overflow.

Mathematical goal

For each positive integer below 10000, repeatedly reverse its decimal digits and add. Count candidates that do not reach a palindrome within 50 iterations.

How the C code works

The helper `reverse` extracts base-ten digits using remainder and division, building the reversed number. `isPalindrome` compares a number with its reversal. For each starting value, the main loop performs up to 50 reverse-and-add operations, clears the `lychrel` flag if a palindrome appears, and increments `score` otherwise.

Correctness and limitations

The outer loop includes 10000 itself (`a<10001`), one beyond the stated strict upper bound. More importantly, repeated reverse-and-add operations can exceed the capacity of `unsigned long long`; unsigned overflow silently changes the sequence. Arbitrary-precision integers are needed for a trustworthy 50-iteration test. The expected candidate count is `...249`, but the program's fixed-width arithmetic may not reproduce it.

Correction to historical C

Correction: GNU MP replaces fixed-width integers to avoid overflow during fifty reverse-and-add attempts; only starting values strictly below 10000 are counted.

36 Totient Maximum — Problem 69

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/totient_maximum_69.c`

Assessment: Totient-ratio search implemented.

Mathematical goal

Euler's totient $\varphi(n)$ counts positive integers at most n coprime to n. Find $n \leq 10^6$ maximizing $n/\varphi(n)$. For distinct prime divisors p of n,

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right).$$

How the C code works

The helper `phi` begins at `res=i` and, whenever a prime factor is found, updates `res -= res/p`, removing repeated copies of that factor from the working integer. `main` loops over candidate `n` values from 2 up to one million, calculates the floating-point ratio, and records the best position in `maxpos`.

Correctness and limitations

The factorization-based totient computation is appropriate, and the expected maximizing integer is `...0510`. The code recomputes `phi(n)` after deciding the ratio improved, doubling work for those iterations. Its diagnostic print compares two equal values because `maxval` was just updated. A product of successive small primes offers a faster mathematical route. The printed maximum remains valid if the helper is evaluated correctly.

Correction to historical C

Correction: The corrected implementation multiplies successive primes while staying at most one million. The original totient factorization was useful but recomputed values unnecessarily.

37 Square Digit Chains — Problem 92

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/square_digit_chains_92.c`

Assessment: Unimplemented.

Mathematical goal

Starting from an integer n, repeatedly replace it with the sum of the squares of its decimal digits. Count starting values below ten million whose chain reaches 89 rather than 1.

How the C code works

The source includes a comment describing the digit-square transformation and example chains. Its `main` contains no calculations, no loop over starting numbers, and no counter; it immediately returns zero.

Correctness and limitations

This is a problem statement preserved as a C placeholder, not an algorithm. An implementation can precompute the destination of each possible digit-square sum and reuse that result across all starting values. There is no output from the current source that answers the problem; the mathematical count is `...1146`.

Correction to historical C

Correction: The formerly empty C program now evaluates repeated decimal digit-square sums and counts exactly those starts below ten million that reach 89.

38 Cross-language algorithm comparisons

The original 37-problem collection has a dated validation baseline for C, Rust, and Gforth. C, Rust, and Factor now have complete 120-answer validation records. A 60-second-per-solver Gforth run passes 115 entries and times out on five. The *Factor programming language* is a concatenative, stack-based language with its own VM; it is neither Gforth nor the GNU `factor` integer-factorization command. The excerpts below reflect the algorithms in this development branch, not the uncorrected historical programs.

The C, Rust, and Gforth algorithms now reside in their individual problem files. Shared libraries contain reusable mathematical and input operations, with no problem-number dispatcher. Problem-specific search state and helper routines stay with their owning algorithm. The code excerpts below identify those owners.

38.1 Problem 9: Special Pythagorean Triplet

The perimeter constraint eliminates one search variable:

$$c = 1000 - a - b, \qquad a < b < c, \qquad a^2 + b^2 = c^2.$$

All versions test the same conditions but spell the computation differently.

C (`solutions/c/special_pythagorean_triplet_9.c`):

```
int c=1000-a-b;
if(c>b&&a*a+b*b==c*c)return a*b*c;
```

Rust (`solutions/rust/src/bin/p9.rs`):

```
let c=1000-a-b;
if c>b&&a*a+b*b==c*c {answer=a*b*c;break;}
```

Gforth (`solutions/forth/p9.fs`, `pythagorean-product`):

```
1000 a - b - to c
c b > a a * b b * + c c * = and
if a b * c * else 0 then ;
```

Factor (`solutions/factor/p9.factor`):

```
1000 a - b - :> c
c b > [
    a sq b sq + c sq = [
        a b * c * answer!
    ] when
] when
```

Unlike the C early return, this initial Factor version continues scanning the fixed candidate range after finding a solution. Both remain bounded searches, but the extra work prevents using these timings as a pure language benchmark.

38.2 Problem 11: Largest Product in a Grid

For each grid coordinate (x, y), the four direction vectors are $(1, 0)$, $(0, 1)$, $(1, 1)$ and $(-1, 1)$. A four-cell window is legal only when its endpoint $(x + 3d_x, y + 3d_y)$ stays in the 20×20 grid:

$$P(x, y, d_x, d_y) = \prod_{k=0}^{3} G[y + kd_y, x + kd_x].$$

Checking the endpoint first eliminates the negative-index undefined behavior in the historical C attempt.

C:

```
const int dx[]={1,0,1,-1},dy[]={0,1,1,1};
int xx=x+3*dx[d],yy=y+3*dy[d];
if(xx<0||xx>=20||yy<0||yy>=20)continue;
long long product=1;
for(int k=0;k<4;k++)
    product*=grid[y+k*dy[d]][x+k*dx[d]];
```

Rust:

```
let dirs=[(1,0),(0,1),(1,1),(-1,1)];
let xx=x+3*dx; let yy=y+3*dy;
if xx<0||xx>=20||yy<0||yy>=20 {continue;}
```

Gforth, using a flat cell array:

```
grid y i dy * + 20 * x i dx * + + cells + @
product * to product
```

Factor, using sequence access:

```
y step dy * + 20 *
x step dx * + +
grid nth result * result!
```

These are excerpts of inner loops; the complete source validates all four directions and all starting points. The Factor and C variants read the committed `tests/data/p011.txt` fixture at runtime. The Rust version embeds it, so I/O work differs.

38.3 Problem 28: Number Spiral Diagonals

For odd side length $s \geq 3$, the four corners on its newest ring have sum

$$\Delta(s) = 4s^2 - 6(s - 1).$$

The final sum is $1 + \Delta(3) + \Delta(5) + \cdots + \Delta(1001)$; no 1001×1001 array is required.

C:

```
for(long long side=3;side<=1001;side+=2)
    sum+=4*side*side-6*(side-1);
```

Rust:

```
for side in (3i64..=1001).step_by(2) {
    sum+=4*side*side-6*(side-1);
}
```

Gforth:

```
i i * 4 * i 1- 6 * - result + to result
```

Factor:

```
ring 2 * 1 + :> side
side sq 4 * side 1 - 6 * -
answer + answer!
```

All four perform a fixed number of ring updates. Their language-specific loop mechanics and VM/process startup can dominate the short computation.

38.4 Problem 39: Integer Right Triangles

The C and Rust implementations test side combinations for all perimeters $p \leq 1000$. Gforth and Factor instead solve the two equations for b:

$$b = \frac{p(p-2a)}{2(p-a)}, \qquad c = p - a - b,$$

then accept only exact integer divisions satisfying $a < b < c$. This reduces a nested side search to one candidate b per pair (p, a).

C:

```
int c=p-a-b;
if(a*a+b*b==c*c)count++;
```

Rust:

```
let c=p-a-b;
if a*a+b*b==c*c {count+=1;}
```

Gforth:

```
j j i 2 * - * to numerator
2 j i - * to denominator
numerator denominator mod 0= if
    numerator denominator / to b
    ...
then
```

Factor:

```
p p a 2 * - * :> numerator
p a - 2 * :> denominator
numerator denominator mod zero? [
    numerator denominator /i :> b
    p a - b - :> c
    a b < b c < and [ count 1 + count! ] when
] when
```

The Gforth excerpt includes an ellipsis where the full source applies its positivity and ordering tests. Differences in asymptotic work mean that these four implementations must not be ranked as if they executed identical algorithms.

38.5 Exact arithmetic: Factor versus C and Gforth

For problem 16 the Factor code expresses its entire big-integer computation as a composition of existing arbitrary-precision words:

```
: p16 ( -- n ) 2 1000 ^ digit-sum ;
```

Problem 20 likewise composes a range product with decimal digit summation. C uses GMP and Gforth uses a project-specific base-10000 digit library. These implementations trade library code size, native compilation, data representation, and startup costs against different amounts of hand-written algorithm code.

38.6 Profiling interpretation

The runner executes each selected binary or VM script *once*, without warming up; it checks that same invocation's exact decimal output against `tests/answers.json`. The wall time includes startup and output capture, but not compilation. It records process version, source revision and selected coverage; no error bars or confidence intervals are inferred from one measurement. Only complete passing 120-problem language suites get total-suite bars. In the current Gforth run, the five timed-out entries are marked explicitly and no Gforth total-suite bar is drawn.

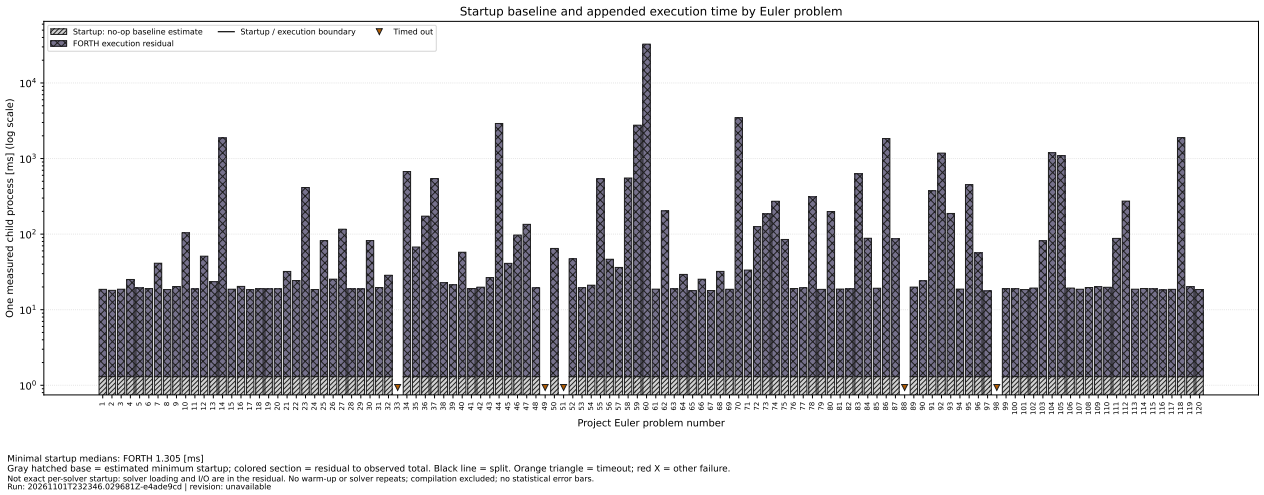


Figure 1: The recorded profile run preceding the ownership refactor. Passing solvers have measured bars; timeouts use orange triangles, while wrong answers and runtime errors would use red crosses. The run identifier printed in the figure ties it to its CSV and metadata.

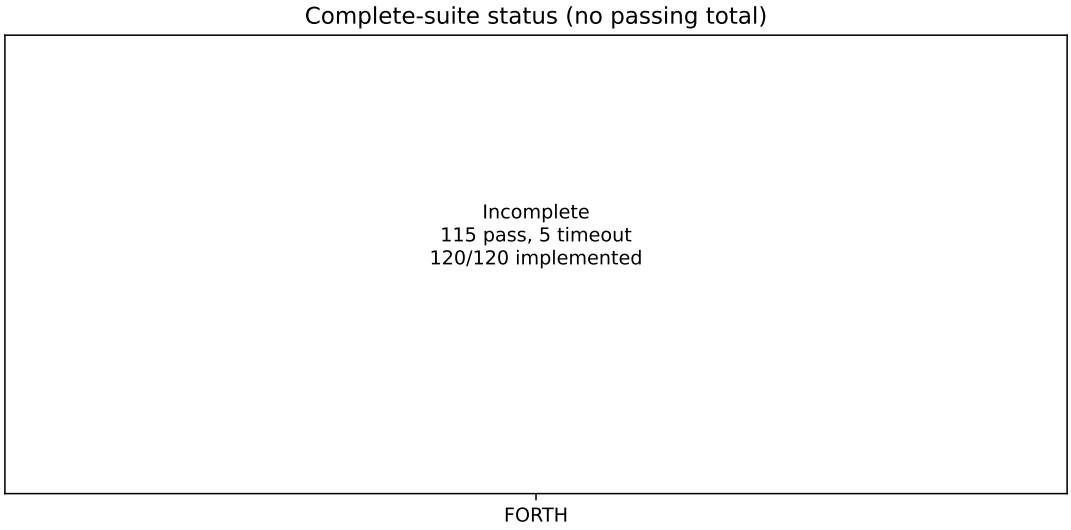


Figure 2: Complete-suite status for the same run. An incomplete suite receives no duration bar, so timeout work cannot be mistaken for a passing total.

39 Mathematical visualization atlas

This atlas supports algorithm explanations with actual data where available, and with explicitly labelled reduced examples elsewhere. Plots are generated by `make plots-math` and embedded only when the corresponding PDF exists. Every figure has an SVG, PNG and PDF counterpart under `build/math_plots/`. The complete topic-to-file map is recorded in `docs/visualization_index.md`. None of these figures is a measurement of algorithm runtime.

39.1 Integer geometry and spirals

39.2 Grids and dynamic programming

39.3 Discrete counting and networks

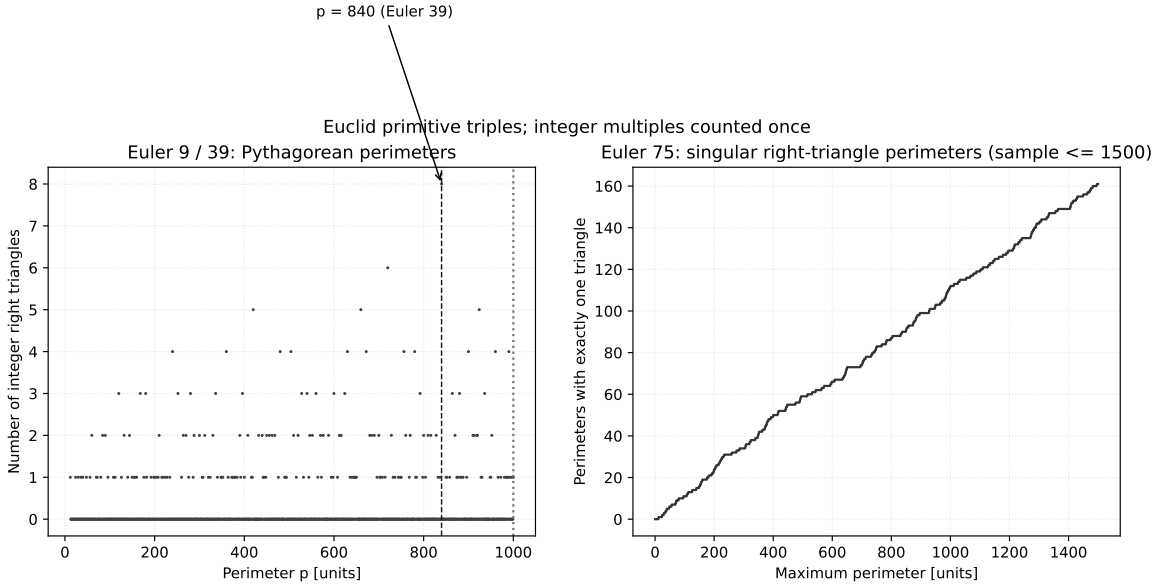


Figure 3: Project Euler 9, 39, 75: Euclid's formula enumerates right-triangle perimeter counts; the dashed marker gives the full problem 39 winner $p = 840$. Singular-count growth is sampled only to $p = 1500$.

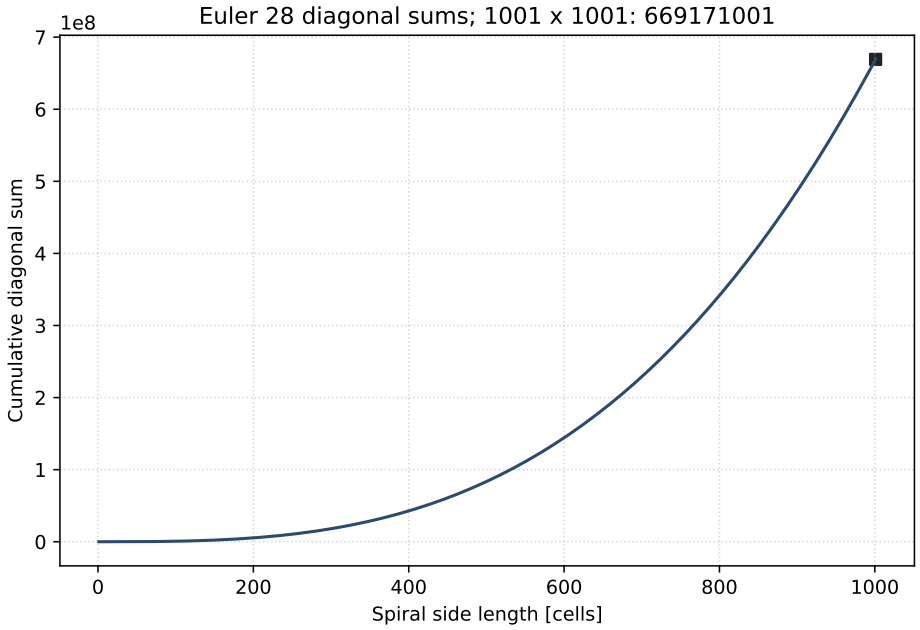


Figure 4: Project Euler 28: Sum of the four corners of each odd-width ring through the full 1001×1001 target.

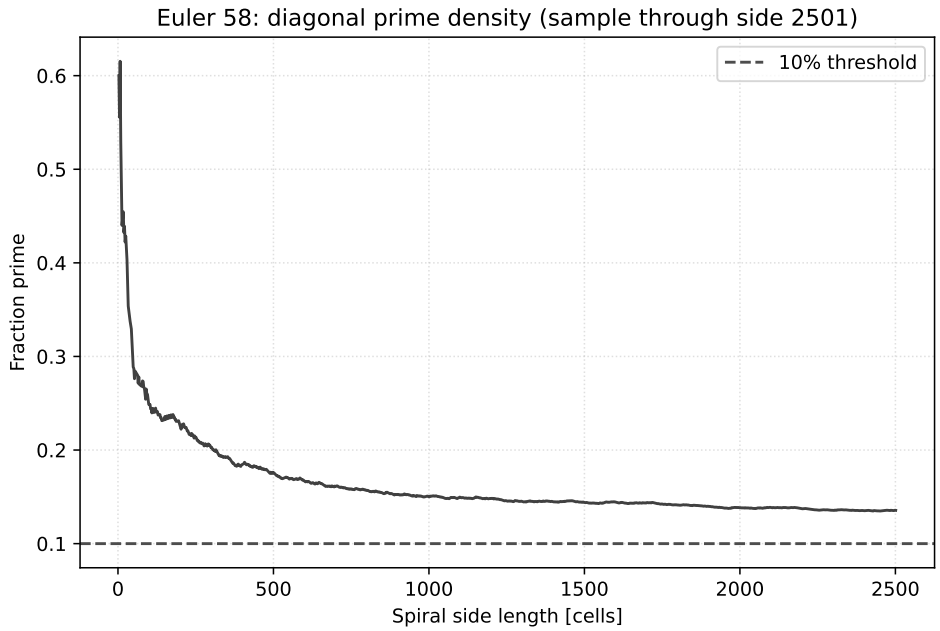


Figure 5: Project Euler 58: Prime density on spiral diagonals sampled through side length 2501, with a 10-percent reference line.

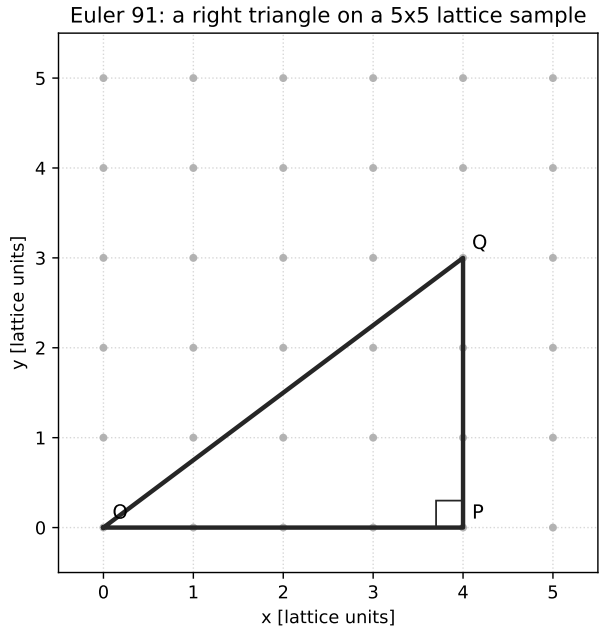


Figure 6: Project Euler 91: Lattice-coordinate geometry on a small grid; the full counting problem is not implemented here.

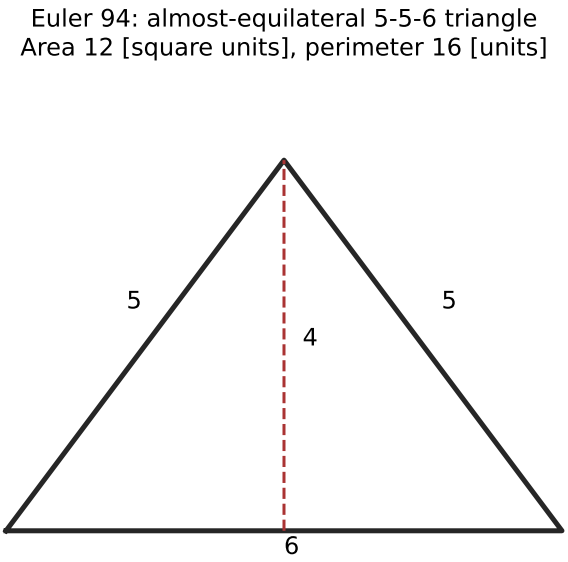


Figure 7: Project Euler 94: The $5, 5, 6$ example has an integral area and differs from an equilateral triangle by one unit.

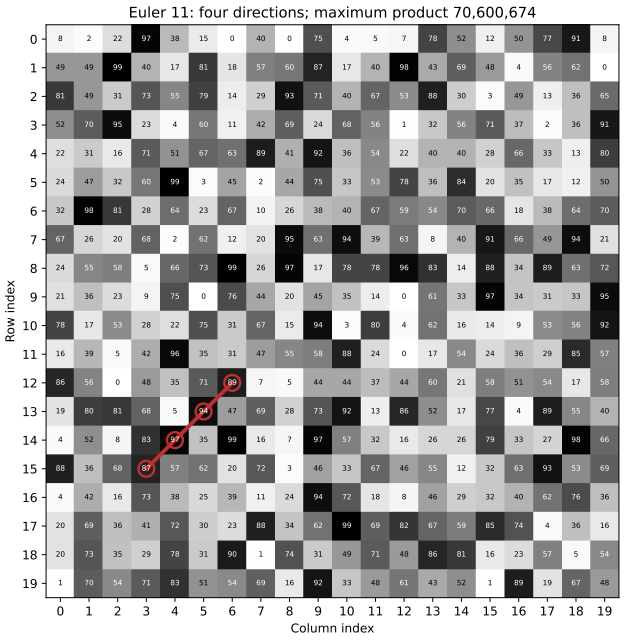


Figure 8: Project Euler 11: Actual 20×20 input grid from the repository. The highlighted window produces the verified maximum product $70,600,674$.

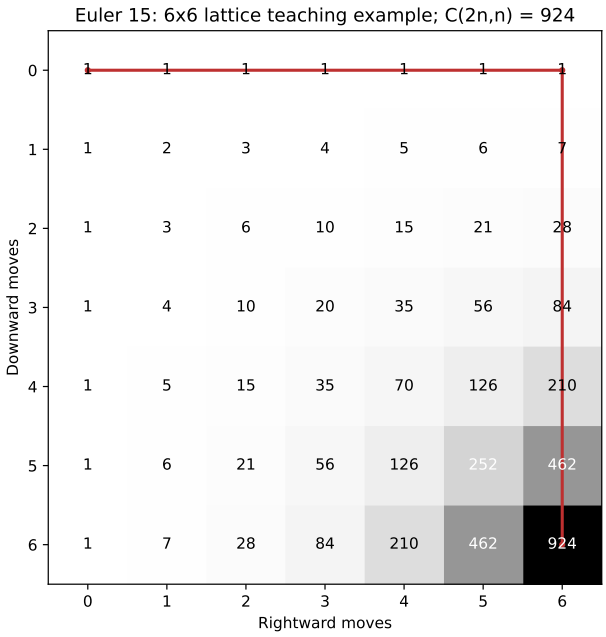


Figure 9: Project Euler 15: Binomial coefficients and a sample six-by-six lattice path. The chart is not the full problem input.

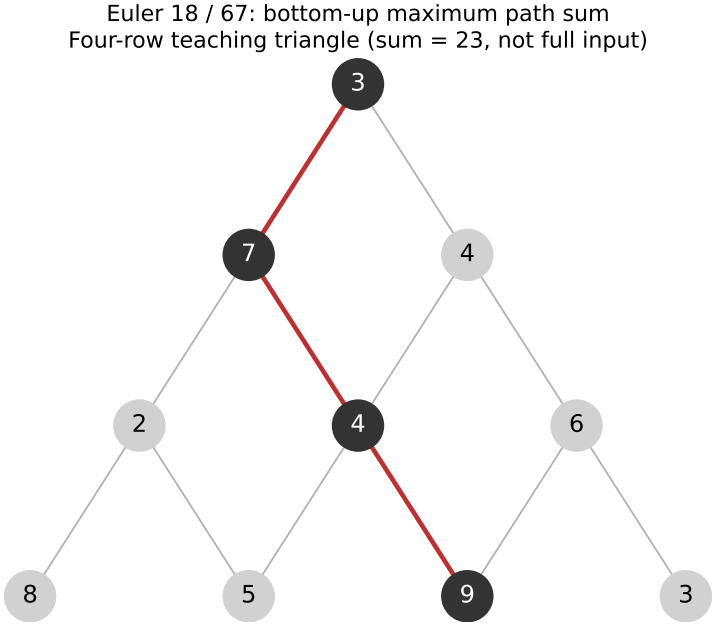


Figure 10: Project Euler 18, 67: A reduced four-row triangle shows how bottom-up dynamic programming identifies the maximum path sum.

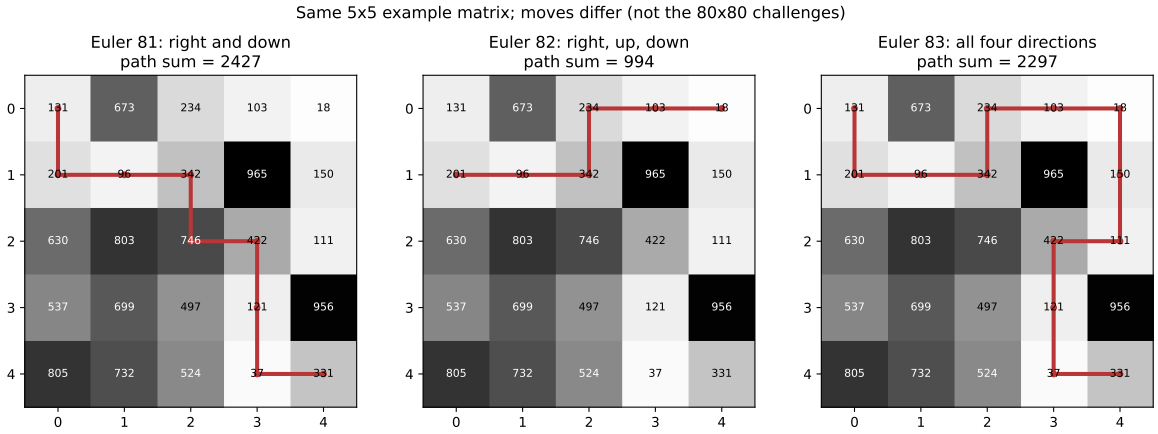


Figure 11: Project Euler 81, 82, 83: Three shortest-path models share one five-by-five example matrix but allow different moves. The full-sized input matrices are not supplied.

Euler 96: Sudoku teaching example
Bold given digits; gray digits filled by backtracking

4	8	**3**	9	**2**	1	**6**	5	7
9	6	7	**3**	4	**5**	8	2	**1**
2	5	**1**	**8**	7	**6**	**4**	9	3
5	4	**8**	**1**	3	**2**	**9**	7	6
7	2	9	5	6	4	1	3	**8**
1	3	**6**	**7**	9	**8**	**2**	4	5
3	7	**2**	**6**	8	**9**	**5**	1	4
8	1	4	**2**	5	**3**	7	6	**9**
6	9	**5**	4	**1**	7	**3**	8	2

Figure 12: Project Euler 96: A solved nine-by-nine example, distinguishing givens from cells filled by backtracking.

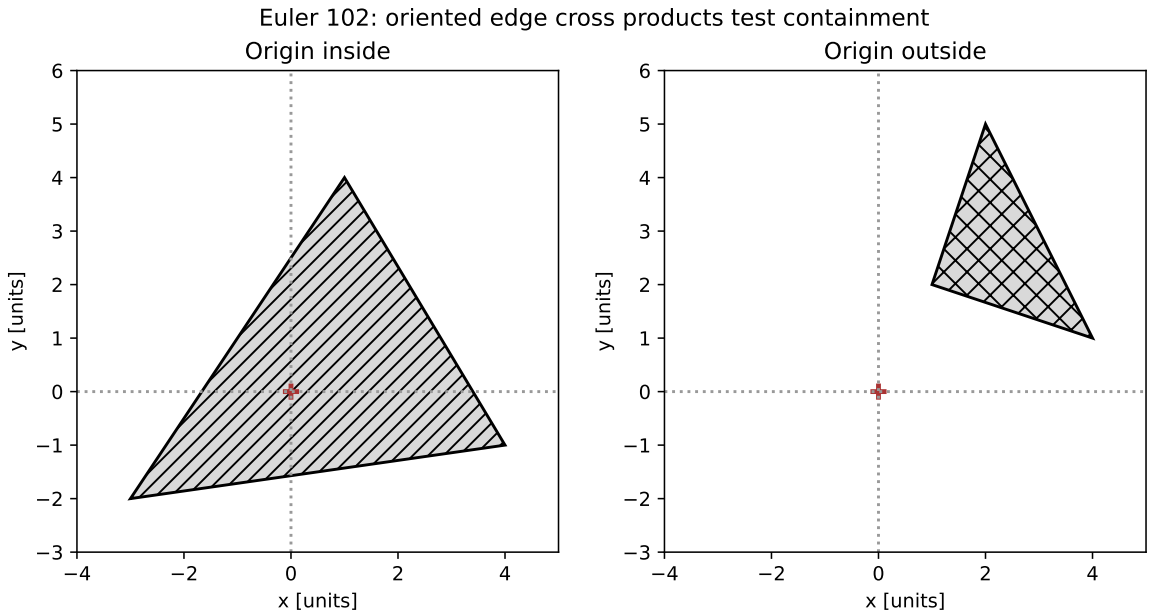


Figure 13: Project Euler 102: The signs of three oriented cross products decide whether the origin is inside a triangle.

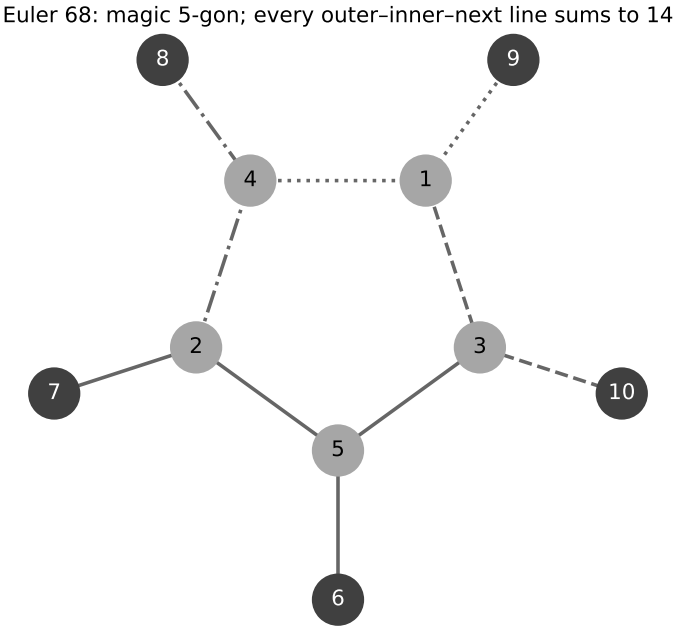


Figure 14: Project Euler 68: One valid five-gon configuration whose five triples have the same sum.

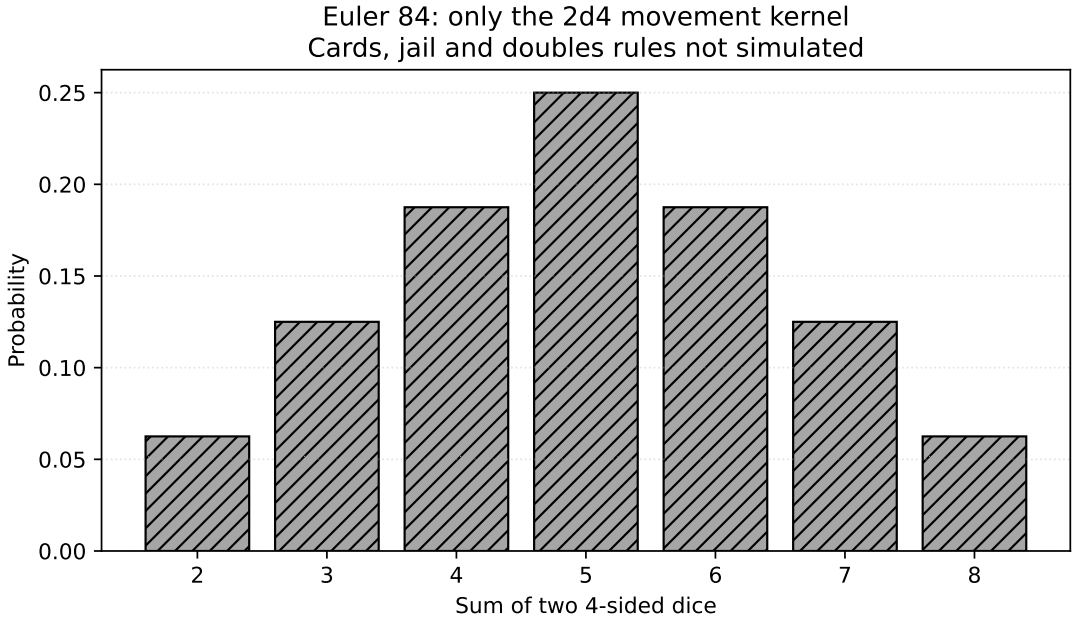


Figure 15: Project Euler 84: The probability mass function of two four-sided dice. This is only the movement kernel, not Monopoly's complete Markov model with jail and card rules.

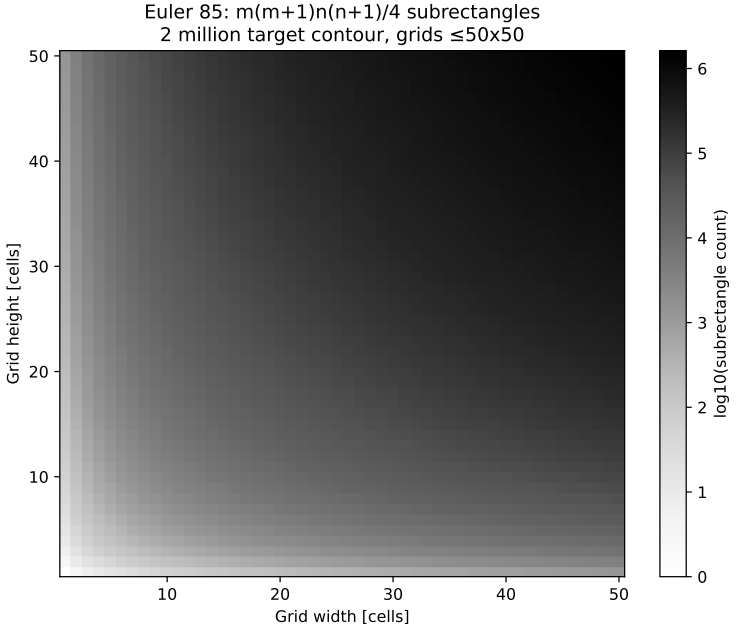


Figure 16: Project Euler 85: The number of rectangular subgrids across width and height, with a target contour at two million.

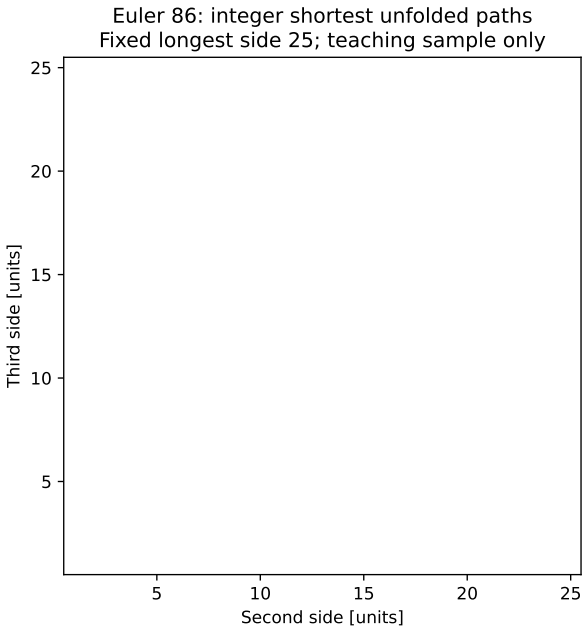


Figure 17: Project Euler 86: Integer-length unfolded surface routes for a restricted set of cuboids.

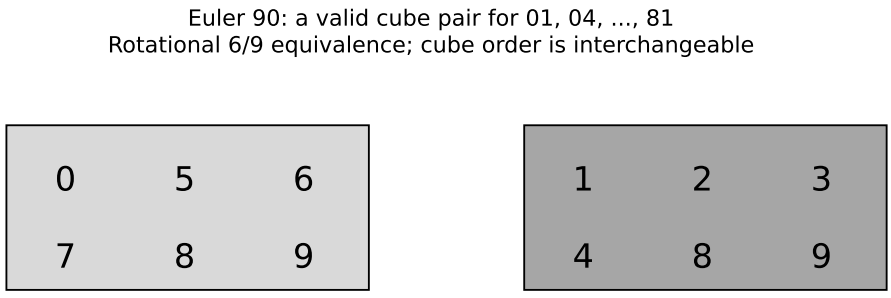


Figure 18: Project Euler 90: A valid digit placement on two cubes, where six and nine can be exchanged by rotating the cube.

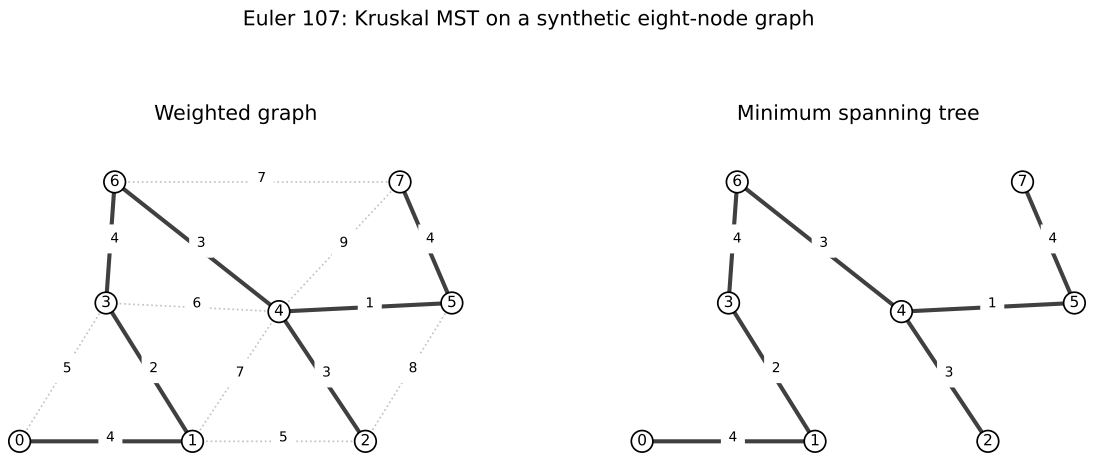


Figure 19: Project Euler 107: Kruskal's minimum-spanning-tree procedure, shown on an explicitly synthetic graph.

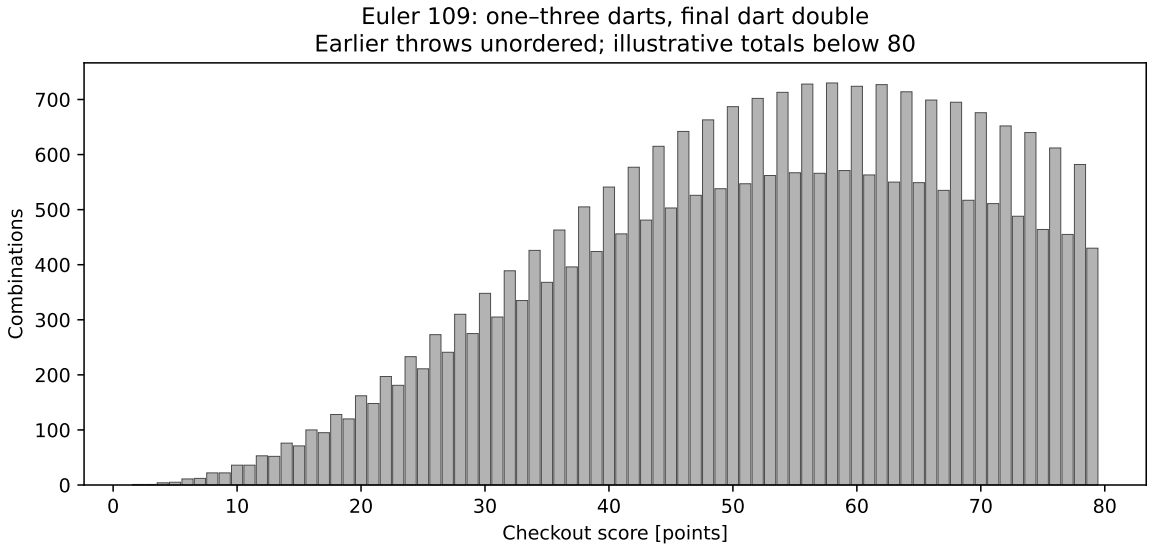


Figure 20: Project Euler 109: Distinct checkout combinations under score 80, with an obligatory final double.

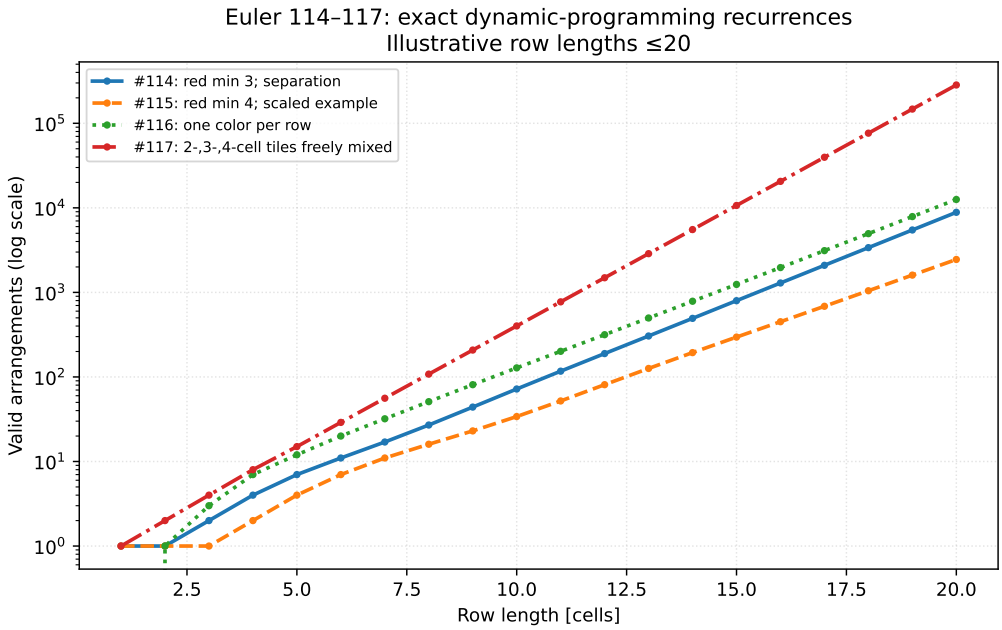


Figure 21: Project Euler 114, 115, 116, 117: Four exact dynamic-programming recurrences on short rows; the second curve is a scaled illustration, not the full problem-115 threshold.